Exam AWS Certified Developer - Associate DVA-C02 All Questions

View all questions & answers for the AWS Certified Developer - Associate DVA-C02 exam

Go to Exam

EXAM AWS CERTIFIED DEVELOPER - ASSOCIATE DVA-C02 TOPIC 1 QUESTIO...

Exam question from Amazon's AWS Certified Developer - Associate DVA-C02
Question #: 102
Topic #: 1
[All AWS Certified Developer - Associate DVA-C02 Questions]

A social media application uses the AWS SDK for JavaScript on the frontend to get user credentials from AWS Security Token Service (AWS STS). The application stores its assets in an Amazon S3 bucket. The application serves its content by using an Amazon CloudFront distribution with the origin set to the S3 bucket.

The credentials for the role that the application assumes to make the SDK calls are stored in plaintext in a JSON file within the application code. The developer needs to implement a solution that will allow the application to get user credentials without having any credentials hardcoded in the application code.

Which solution will meet these requirements?

A. Add a Lambda@Edge function to the distribution. Invoke the function on viewer request. Add permissions to the function's execution role to allow the function to access AWS STS. Move all SDK calls from the frontend into the function.

B. Add a CloudFront function to the distribution. Invoke the function on viewer request. Add permissions to the function's execution role to allow the function to access AWS STS. Move all SDK calls from the frontend into the function.

C. Add a Lambda@Edge function to the distribution. Invoke the function on viewer request. Move the credentials from the JSON file into the function. Move all SDK calls from the frontend into the

function.

D. Add a CloudFront function to the distribution. Invoke the function on viewer request. Move the credentials from the JSON file into the function. Move all SDK calls from the frontend into the function.

Show Suggested Answer

by MrTee at *April 22, 2023, 3:30 p.m.*

Disclaimers:
- ExamTopics website is **not** related to, affiliated with, endorsed or authorized by Amazon.
- Trademarks, certification & product names are used for reference only and belong to Amazon.

Comments

csG13 Highly Voted 2 years, 3 months ago

Selected Answer: A

The answer is A. Here is a reference directly from AWS docs:

"If you need some of the capabilities of Lambda@Edge that are not available with CloudFront Functions, such as network access or a longer execution time, you can still use Lambda@Edge before and after content is cached by CloudFront."

Since the requirement is to access the STS service, network access is required. Therefore, it can't be Cloudfront functions. Also, as a side note it's worth to mention that Cloudfront functions can only execute for up to 1ms. Apparently this isn't enough to fetch user creds (tokens) from STS.

The table in the following link summarises the differences between Cloudfront functions and Lambda@edge

https://aws.amazon.com/blogs/aws/introducing-cloudfront-functions-run-your-code-at-the-edge-with-low-latency-at-any-scale/

upvoted 14 times

MrTee Highly Voted 2 years, 5 months ago

Selected Answer: B

The difference between A and B is the SDK for Javascript in use here; Lambda@Edge functions can be written in a variety of programming languages, including Node.js, Python, and Java, while CloudFront functions are written in JavaScript.

upvoted 5 times

Cloud_Cloud 2 years, 5 months ago

Now one problem is lambda function can not perform AWS STS command

upvoted 1 times

eboehm 2 years, 3 months ago

After rereading the last part of the question. It doesnt mention that it must remain written in Javascript, but does seem using AWS STS is a requirement so I think I would stick with A being the answer

upvoted 1 times

sumanshu Most Recent 1 year, 3 months ago

Selected Answer: A

B) Eliminated - CloudFront functions are lightweight JavaScript functions designed for simple HTTP request and response manipulations (e.g., header rewrites, URL rewrites). They cannot access AWS services like STS or assume roles

C/D) Eliminated - Moving the credentials from the JSON file to the Lambda@Edge function does not eliminate the core issue of hardcoding credentials

upvoted 1 times

65703c1 1 year, 4 months ago

Selected Answer: A

A is the correct answer.

upvoted 1 times

rrharris 1 year, 7 months ago

Selected Answer: A

Why A is Correct:
Lambda@Edge for Secure Credential Management: Lambda@Edge allows you to run Lambda functions in response to CloudFront events. By using Lambda@Edge, the developer can securely manage credentials by keeping them out of the client-side code.

Invoking on Viewer Request: Invoking the Lambda@Edge function on viewer requests ensures that the credential generation happens in real-time, securely, and as needed, without exposing any sensitive information.

Execution Role with STS Access: Assigning the Lambda function an execution role with permissions to access AWS STS (Security Token Service) enables the function to securely request temporary, limited-privilege credentials on behalf of the client.

Moving SDK Calls to Lambda@Edge: Transferring all AWS SDK calls from the frontend to the Lambda@Edge function prevents exposing any credentials in the frontend code, enhancing security.

upvoted 3 times

SerialiDr 1 year, 8 months ago

Selected Answer: A

A. Lambda@Edge allows you to run Lambda functions in response to CloudFront events. By using a Lambda@Edge function, you can securely handle the process of obtaining credentials from AWS STS without exposing them in the client-side application code. The function's execution role can be granted the necessary permissions to interact with AWS STS, and SDK calls can be made from within this server-side environment. This approach centralizes credential management and AWS interactions in a more secure, server-side context.

upvoted 4 times

LR2023 1 year, 10 months ago

I think i will also go with A as cloudfront functions can only read authorization headers from the viewer request if it sees the authorization header request. And Clouf front functions has no access to internet.

upvoted 2 times

Baba_Eni 2 years ago

Selected Answer: A

I will go for A, check the link below, Cloudfront functions are just within Cloudfront, hence, they DONT HAVE NETWORK ACCESS. Network access is required to make a call to AWS STS.

https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/edge-functions.html

upvoted 1 times

MG1407 2 years, 1 month ago

The answer is B. I was in agreement with csG13 until a further research into the JavaScript SDK and STS. Found the following: https://docs.aws.amazon.com/AWSJavaScriptSDK/v3/latest/clients/client-cloudfront/classes/stsclient.html.
Since the question states Js SDK and STS the answer is B.

upvoted 1 times

FunkyFresco 2 years, 4 months ago

Selected Answer: A

Option A.

upvoted 1 times

zodraz 2 years, 4 months ago

Selected Answer: A

https://www.examtopics.com/discussions/amazon/view/89838-exam-aws-certified-developer-associate-topic-1-question-361/

upvoted 2 times

vic614 2 years, 5 months ago

Selected Answer: A

Cloud front function doesn't have network access, it has to be lambda @ edge

I |

upvoted 2 times